

학습목표 기반 구성

PART 2. Private LLM 서빙

조직 내부 인프라(온프레미스 서버 또는 독립된 프라이빗 클라우드 VPC)에 오픈소스 대형 언어 모델을 직접 배포하여 서비스 형태로 운영하는 것을 의미

학습 목표

- LLM 서빙의 병목이 어디서 발생하는지 설명할 수 있다
- vLLM의 PagedAttention이 왜 중요한지 이해한다
- 양자화와 배포 전략의 선택 기준을 설명할 수 있다

도입

"Part 1에서 GPU 서버를 잘 갖췄다고 가정. 그런데 막상 서비스를 열어보니 사용자가 물리면 응답이 느려지고, 어떤 요청은 아예 처리가 안 되는 상황이 생깁니다.
왜 그럴까요? 이걸 인프라가 부족해서가 아니라, GPU 메모리를 쓰는 방식 자체에 병목이 있기 때문입니다.
오늘은 이 병목이 정확히 어디서 생기고, 어떻게 줄일 수 있는지 살펴보겠습니다."

2-1 서버의 병목 지점

LLM 한 번의 응답 생성은 크게 두 단계로 나뉜다.

단계	설명	특징
Prefill	입력 프롬프트 전체를 한 번에 병렬 처리하는 첫 단계	GPU 연산을 병렬로 꽉 채워 써 상대적으로 빠름
Decode	토큰을 하나씩 순차 생성하는 단계	매번 가중치와 KV 캐시를 다시 읽어야 하므로 서버 속도의 실질적 병목

KV 캐시

트랜스포머 기반 LLM이 텍스트를 생성(Decoding)할 때, 이전에 계산했던 토큰들의 **Key(K)**와 **Value(V)** 벡터를 버리지 않고 **GPU VRAM**에 저장해 두어 중복 연산을 건너뛰는 최적화 메모리 기법

- a. KV 캐시가 없는 상태: '매 페이지마다 책 1페이지부터 다시 읽음'
- b. KV 캐시가 있는 상태: '읽은 내용을 요약 메모해 둠'

Throughput(처리량)

단위 시간당 처리 요청 수.

Latency(지연시간)

요청 하나가 끝나는 데 걸리는 시간.

기초 키워드 명사부터 알자 = 생성형 AI 사용

서빙(Serving)

사용자의 요청(입력)을 받아 -> 학습된 모델의 추론(Inference) 연산을 수행 -> 그 결과를 실시간 응답(출력)으로 반환하는 **소프트웨어 및 인프라 파이프라인 전체**를 의미.

(예시) 주방장(GPU연산)이 쉬지 않도록 -> 손님 주문(모델 가중치 및 KV 캐시 배치 처리)을 묵고 -> 재료 통로(GPU VRAM 대역폭)의 정체를 뚫어 -> 손님 테이블(사용자 응답)로 끊임 없이 음식을 내보내는 것.

병목 해결 방법

서빙에서 발생하는 3대 핵심 병목 지점

1. 연산 병목 (Compute-bound): 프리필(Prefill) 단계

원인

사용자가 보낸 긴 프롬프트를 한 번에 병렬 연산(Matrix Multiplication, GEMM)할 때 발생.

병목 특성

GPU의 연산 성능(TFLOPS) 자체가 한계에 도달하여 응답 시작 시간(TTFT, Time To First Token)이 지연.

2. 메모리 대역폭 병목 (Memory-bandwidth-bound): 디코딩(Decoding) 단계

원인 LLM은 토큰을 하나씩 순차적으로 생성(Autoregressive)함. 토큰 단 **1개**를 만들 때마다 수십 GB에 달하는 모델 가중치 전체와 이전 토큰들의 KV 캐시를 VRAM에서 연산 코어로 퍼 올려야 함.

병목 특성 연산 코어는 여유가 넘치는데, 메모리에서 데이터를 퍼 나르는 속도(GB/s)가 느려 토큰 생성 속도(TPOT, Time Per Output Token)가 떨어짐.

3. 메모리 용량 병목 (Capacity-bound): 동시성(Concurrency) 한계

원인 동시 접속자(Batch)가 늘어나고 문맥(Context Window)이 길어질수록, 각 사용자 세션의 **KV** 캐시가 VRAM을 기하급수적으로 차지함.

병목 특성 모델 파라미터를 올리고 남은 잉여 VRAM이 금방 바닥나며 OOM(Out Of Memory)이 발생하거나, 동시 처리 가능한 사용자 수(Throughput)가 제한됨.

서빙 엔진의 병목 해결 방식

현대 LLM 서빙 프레임워크(vLLM 등)는 이러한 병목을 해결하기 위해 설계된 도구

메모리 단편화 해결 **PagedAttention**을 통해 OS의 가상 메모리 기법처럼 KV 캐시를 잘게 쪼개어 할당함으로써 VRAM 낭비를 없게 함.

유틸 코어 활용 연속 배칭(Continuous Batching / Iteration-level Batching)으로 먼저 끝난 요청의 자리에 새 요청을 즉시 끼워 넣어 GPU 연산 자원이 쉬지 않도록 유지함.

대역폭 부담 완화 양자화(Quantization, FP8/INT4)를 적용해 메모리에서 퍼 날라야 하는 가중치 크기 자체를 줄여 대역폭 정체를 완화.

비유로 설명하기:

Prefill은 "시험 문제지를 한 번에 쪽 읽는 것"이고,
Decode는 "답을 한 글자씩 손으로 눌러쓰는 것".
읽는 건 빠르지만, 쓰는 건 한 글자씩이라 오래 걸린다.
이미 쓴 글자를 계속 다시 확인해야 한다면(KV 캐시) 문장이 길어질수록 더 느려짐.

2-2 vLLM과 PagedAttention

개념	설명
PagedAttention	OS의 페이지 메모리 관리 방식을 KV 캐시에 적용해, 필요한 만큼만 블록 단위로 할당
Continuous Batching	끝난 요청 자리를 즉시 새 요청으로 채워 GPU 유휴 시간을 최소화
(기존 방식)	요청마다 KV 캐시 공간을 미리 통째로 예약해 메모리 낭비가 컸다.
PagedAttention	필요한 블록만 그때그때 할당해 메모리 활용도를 높이고, 더 많은 요청을 동시에 처리할 수 있게 함.
Continuous Batching	한 배치 안에서 먼저 끝난 요청이 있으면 그 자리를 바로 새 요청으로 채워, GPU가 노는 시간을 줄임.

기초 키워드 명사부터 알자 = 생성형 AI 사용

- vLLM** 초고속 LLM 추론 및 서빙 전용 오픈소스 엔진.
운영체제의 가상 메모리 기법을 차용한 **PagedAttention**을 통해 GPU VRAM 낭비를 극도로 줄여 처리량(Throughput)을 기존 대비 2~4배 이상 끌어올린 프레임워크
(예시) 기존 서빙 방식: '대형 버스 주차장에 경차를 주차하여 공간 낭비'
vLLM: '경차가 들어오면 비어있는 작은 칸에 주차 => 물리적으로 떨어져 있어도 하나의 장부(Block Table)로 묶어 차곡차곡 채워 넣는 방식'

비유로 설명하기:

기존 방식 => "식당에서 손님이 몇 명 올지 몰라서 테이블을 통째로 다 예약해두는 것"이고,
PagedAttention => "필요한 만큼만 좌석을 그때그때 배정하는 것"이다. 자리를 미리 다 잡아두면 실제로는 안 쓰는 빈 테이블이 많아지지만, 필요한 만큼만 배정하면 같은 홀에서 훨씬 많은 손님을 받을 수 있다.
Continuous Batching => "한 손님이 식사를 마치자마자 바로 다음 손님을 그 자리에 앉히는 것"에 해당한다.

2-3 양자화와 배포 전략

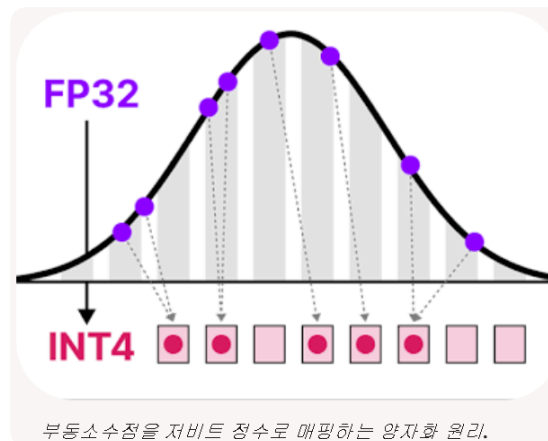
개념	설명
양자화(Quantization)	모델 가중치의 비트 수를 줄여(FP16 → INT8/INT4) 메모리와 속도를 절약
GPTQ / AWQ	정확도 손실을 최소화하며 비트 수를 줄이는 대표적 양자화 기법
Tensor / Pipeline Parallelism	모델이 GPU 한 장에 안 들어갈 때 여러 GPU에 나눠 올리는 방식

- 양자화는 모델 크기를 줄여 더 적은 GPU 메모리로도 서빙할 수 있게 해주지만, 비트 수를 너무 줄이면 응답 품질이 떨어질 수 있어 균형이 필요하다.
- 모델이 하나의 GPU에 다 올라가지 않을 만큼 크다면, Tensor Parallelism(레이어 내부를 나눔)이나 Pipeline Parallelism(레이어 단위로 나눔)으로 여러 GPU에 분산한다.

키워드 명사부터 알자 = 생성형 AI 사용

양자화

모델의 가중치(Weight)와 활성화값(Activation)을 표현하는 고정밀 부동소수점(FP32, FP16/BF16) 데이터를 -> 더 적은 비트 수의 저정밀도 데이터(INT8, FP8, INT4 등)로 압축 및 변환하여 모델 크기를 줄이고 추론 속도를 가속하는 경량화 기술.



(예시) '24색 전문가용 색연필 세트(원본모델)' : 그림(출력 품질)은 완벽하지만 필통(GPU VRAM)이 너무 크고 무거워 다 안들어 감. vs
'6색 기본 색연필 세트(양자화모델)' : 필통 부피가 1/4로 줄어들어 작은 가방(중저가 GPU 1장)에도 다 들어감

비유로 설명하기: 양자화는 "고화질 사진을 약간 압축해서 용량을 줄이는 것"과 비슷.

너무 심하게 압축하면 화질이 눈에 띄게 나빠지듯, 비트 수를 너무 줄이면 모델의 답변 품질도 떨어진다.

Tensor/Pipeline Parallelism은 "혼자 들기엔 너무 무거운 짐을 여러 사람이 나눠 드는 것"에 비유할 수 있다.

정리

"정리하면, LLM 서빙이 느려지는 근본 원인은 Decode 단계에서 토큰을 하나씩 만들며 KV 캐시를 계속 읽어야 하기 때문임.

vLLM의 PagedAttention과 Continuous Batching은 이 메모리 낭비를 줄여 더 많은 요청을 동시에 처리하게 해주고, 양자화와 병렬화는 모델 자체의 크기 문제를 해결하는 방법임."

확인 질문: "오늘 배운 세 가지 기법 중 하나를 골라, 왜 그게 서빙 속도를 개선하는지 한 문장으로 설명해 볼까요?"

확인 질문 설명:

양자화(Quantization)

가중치 데이터의 비트 수를 줄여 디코딩 단계마다 VRAM에서 연산 코어로 퍼 올려야 하는 데이터 전송량을 대폭 줄임으로써, **메모리 대역폭 병목을 해소**해 토큰 생성 속도를 끌어올림.

KV 캐시

"이전 토큰들의 Key와 Value 연산 결과를 VRAM에 저장해 둬으로써, 매 토큰을 생성할 때마다 문맥 전체를 처음부터 다시 읽는 중복 연산($O(N^2) \rightarrow O(N)$)을 없애 속도를 개선함."

vLLM (PagedAttention & 연속 배칭)

"KV 캐시를 블록 단위로 쪼개어 메모리 낭비를 제거하고 빈자리에 다음 요청을 즉시 투입함으로써, GPU 연산 코어의 유휴 시간(Idle)을 없애 전체 처리량을 극대화 함."

다음 파트 예고: "이렇게 빠르게 서빙하는 구조까지 만들었다면, 이제 남은 건 이 시스템을 안전하게, 그리고 문제없이 운영하는 것입니다. LLMOps로 넘어가겠습니다."